Boardroom: Every Function Explained

A reference to every function in the application source (tests excluded), grouped by folder and file. Each entry gives the function, where it is defined and what it does. Functions marked *inside X* are helpers defined within another function.

217 functions in 41 files.

Folder	Role
`server/`	Node API: the front door that checks and routes every request.
`shared/`	Code used by both the browser and the server.
`src/`	The website (React), split into the app shell, components and helper libraries.
`api/`	Vercel serverless wrappers around the server code.
`runner/`	Code that runs inside the Docker sandbox.
`ocr-service/`	The Python server that runs the Qwen2-VL model.
`scripts/`	Developer scripts.

server/

`server/index.mjs`

Entry point of the Node API. Creates the HTTP server, runs the security checks on every request and routes each URL to its handler; also serves the built website.

Function	What it does	Line
`send(res, code, body)`	Writes a JSON response with a status code and no-cache headers.	23
`async requestBody(req, limit = 65_536)`	Reads the request body, refusing it once it passes the size limit, and parses it as JSON.	28
`validatePayload(body)`	Checks a run request: valid language, non-empty source up to 32 KiB, stdin up to 8 KiB, no NUL characters. Returns an error message or null.	40
`async defaultHealth(language)`	Reports whether code can run: local Docker first, Wandbox as a fallback when configured.	50
`async defaultRun(payload)`	Runs a program in Docker if available, otherwise on Wandbox.	60
`createHandler({ run = defaultRun, health = defaultHealth, now = Date.now, })`	Builds the request handler: security headers, host/origin checks, owner key, rate limits, then routes each /api URL; anything else serves the built website.	69
`createApp(options)`	Wraps the handler in a Node HTTP server.	196

`server/ai/ocr.mjs`

Handwriting recognition client: validates PNG captures and forwards them to the local Qwen2-VL OCR service.

Function	What it does	Line
`ocrServiceUrl(env = process.env)`	Handwriting recognition runs on a pretrained vision-language model (Qwen2-VL) in the local ocr-service/. On Vercel there is no sidecar, so OCR_SERVICE_URL must point at a hosted instance.	13
`ocrTimeoutMs(env = process.env)`	A hosted function must answer within 60 seconds; a local CPU model needs longer.	20
`ocrConfiguration(env = process.env)`	Reports whether OCR is available and which model it uses, without exposing secrets.	26
`loadOcrEnvironment()`	Loads the private .env file into the server's settings at start-up.	35
`async readLimited(response, limit = 256 * 1024)`	Read a JSON response body, refusing anything larger than the limit.	40

Function	What it does	Line
`fail(message, status = 400, code = 'invalid_ocr_request')`	Creates an error with an HTTP status and a code the browser can recognise.	51
`validateOcrImages(body)`	Checks each image is a real PNG, at most 2 MiB and 2048 x 2048, and returns the raw base64.	55
`validateTranscription(value, images, { compact = false } = {})`	Checks the model's answer: line counts, line lengths, flags and total size stay within the runner limits.	79
`async recognizeHandwriting(body, { env = process.env, fetchImpl = fetch,)`	Main OCR call: validates images, posts them to the OCR service with a timeout, validates the result and marks it as needing teacher confirmation.	110
`report(event, fields)` `inside recognizeHandwriting`	Logs a safe diagnostic (model, duration, status) without images or text.	117
`failure(message, status, code, fields)` `inside recognizeHandwriting`	Creates an error that also carries the logged diagnostic.	124

`server/ai/snippet.mjs`

Snippet mode client: asks the model to add boilerplate around loose C++ lines and checks the teacher's lines were kept.

Function	What it does	Line
`fail(message, status = 400, code = 'ocr_snippet_invalid')`	Creates an error with an HTTP status and code.	6
`normalize(line)`	Trims a line and collapses repeated spaces so indentation differences are ignored.	10
`checkKeepsLines(original, wrapped)`	The model may add boilerplate but must not touch the teacher's lines. Every non-blank original line has to appear in the result, unchanged apart from spacing, and in the same order. Returns the first line that does not.	14
`validateSnippet(body)`	Checks the snippet request: C++ only, non-empty, at most 32 KiB.	28
`async prepareSnippet(body, { env = process.env, fetchImpl = fetch,)`	Sends loose lines to the model's /wrap endpoint, validates the program and adds a warning naming any line the model changed.	36
`report(event, fields)` `inside prepareSnippet`	Logs a safe diagnostic for the snippet request.	42
`failure(message, status, code, fields)` `inside prepareSnippet`	Creates an error that carries the logged diagnostic.	47

`server/execution/hosted-runner.mjs`

Runs code on the Wandbox web service when Docker is not available (hosted mode).

Function	What it does	Line
`hostedRunnerConfiguration(env = process.env)`	Reads the Wandbox address and compiler names from settings.	14
`compilerFor(language, env)`	Picks the Wandbox compiler name for C++ or Python.	26
`async hostedRunnerHealth({ env = process.env, fetchImpl = fetch, language = 'c' } = {})`	Asks Wandbox whether the configured compiler exists.	28
`validateHostedProgram(body)`	Checks a run request before it is sent to Wandbox.	41
`diagnosticLines(output, language = 'c')`	Extracts line numbers and messages from compiler or Python output.	51
`async runHostedProgram(body, { env = process.env, fetchImpl = fetch,)`	Sends the program and the teaching headers to Wandbox and converts its reply into the same result shape as Docker.	68

`server/execution/sandbox.mjs`

Runs code in a fresh, locked-down Docker container through the docker command line.

Function	What it does	Line
`command(args, { input, timeout = 4000, limit = 100_000 } = {})`	Runs a docker command with input, a time limit and an output limit, killing it if either is exceeded.	6
`collect(kind, chunk)` `inside command`	Adds a chunk of output and stops the process if the total is too large.	17
`async sandboxHealth()`	Checks Docker is running and the runner image exists.	40
`async runSandbox(payload)`	Starts a new isolated container (no network, read-only, memory/CPU/process limits), runs the program and always removes the container.	50

`server/hosted/hosted-api.mjs`

Shared helpers for the Vercel serverless functions: origin and owner-key checks, JSON parsing and errors.

Function	What it does	Line
`sendJson(res, status, value)`	Writes a JSON response from a Vercel function.	1
`authorize(req, res, { open = false } = {})`	open: true for the health check, which answers everyone (it reports whether the caller holds the owner key).	12
`jsonBody(req, maxBytes)`	Reads and size-checks a Vercel request body.	30
`apiError(res, error)`	Turns an error into a safe JSON error response.	46

`server/lessons/github-snapshots.mjs`

Lesson archive stored in a GitHub repository through the Git Data API (hosted mode).

Function	What it does	Line
`fail(message, status = 503, githubStatus)`	Creates an error for a GitHub archive problem.	5
`pathEncode(value)`	URL-encodes each part of a file path.	7
`githubSnapshotConfiguration(env = process.env)`	Reads and validates the archive repository, branch and token settings.	9
`createGithubSnapshotStore({ env = process.env, fetchImpl = fetch, now = ()`	Each request uses GitHub as the source of truth; no disk, background jobs, or warm-function memory is required for durability.	23
`async request(path, { method = 'GET', body, raw = false } = {})` `inside createGithubSnapshotStore`	Makes one authenticated GitHub API call.	26
`async head()` `inside createGithubSnapshotStore`	Gets the latest commit of the archive branch.	52
`async readFileAt(id, file, sha)` `inside createGithubSnapshotStore`	Reads one file of a saved lesson at a given commit.	62
`async syncStatus()` `inside createGithubSnapshotStore`	Reports archive status (GitHub needs no background sync).	68

`server/lessons/snapshot-bundle.mjs`

Builds the replay bundle saved with each lesson: frozen runner, vendored headers and a SHA-256 manifest.

Function	What it does	Line
`hash(bytes)`	SHA-256 fingerprint of a file's bytes.	9
`includes(source)`	Finds the #include lines in the source.	11
`readAsset(name)`	Reads a file from the runner folder.	18
`async snapshotBundle(scene, program)`	Builds all replay files for a lesson: source, frozen runner, needed headers, Dockerfile, build script and the hash manifest.	20

Function	What it does	Line
`async vendor(name)` `inside snapshotBundle`	Adds one teaching header, and the headers it depends on, to the bundle.	22

`server/lessons/snapshot-replay.py`

Script saved inside each lesson that verifies the bundle and replays it in Docker without the app.

Function	What it does	Line
`verify()`	Recomputes every file's SHA-256 and compares it with the manifest.	12
`main()`	Verifies the bundle, then builds and runs it in a locked-down Docker container.	38

`server/lessons/snapshot-sync.mjs`

Pushes the local lesson Git repository to a remote, retrying in the background.

Function	What it does	Line
`createSnapshotSync({ git, hasHistory })`	Creates the background push loop for the local lesson repository.	7
`async ref(name)` `inside createSnapshotSync`	Name of the private Git ref that remembers the last successful push.	12
`async status()` `inside createSnapshotSync`	Reports local versus remote sync state.	17
`async sync()` `inside createSnapshotSync`	Pushes new lesson commits to the remote without forcing.	31
`queue()` `inside createSnapshotSync`	Schedules a sync without running two at once.	59

`server/lessons/snapshots.mjs`

Lesson archive on disk: validates a saved board and commits it to a local Git repository.

Function	What it does	Line
`invalid(message, status = 400)`	Creates a validation error.	18
`snapshotProgram(scene)`	Derives the source and input from a saved board, requiring confirmed OCR for handwriting.	21
`async prepareSnapshot(body, now = new Date()`	Validates a save request and builds the lesson files and folder name.	64
`createSnapshotStore({ root = resolve(APP, process.env.SNAPSHOT_DIR || 'snapshots')`	Creates the on-disk lesson store: save, list, read, compare headers and sync.	84
`async git(args, timeout = 8000)` `inside createSnapshotStore`	Runs a git command in the lesson repository with a timeout.	87
`async hasHistory()` `inside createSnapshotStore`	Checks whether the lesson repository exists yet.	92

`server/security/owner-key.mjs`

Owner-key check that makes the API private to the key holder.

Function	What it does	Line
`digest(value)`	The website is public, but the API (running code, OCR, snippets, lessons) answers only the owner. The owner key lives in the server's private settings (BOARDROOM_OWNER_KEY) and is typed once in the owner's browser, never built into the website. Without a configured key the API stays open (local development).	6

Function	What it does	Line
`ownerKeyConfigured(env = process.env)`	True when BOARDROOM_OWNER_KEY is set, i.e. the API is private.	8
`hasOwnerKey(req, env = process.env)`	True when the request carries the correct key (or no key is configured).	12

`server/security/rate-limit.mjs`

Per-visitor and server-wide request limits.

Function	What it does	Line
`createRateLimiter({ perClient, total, now = Date.now, windowMs = WINDOW_MS })`	Creates a limiter with a per-visitor and a server-wide budget per minute; returns an allow(client) function.	6
`prune(time)` inside `createRateLimiter`	Forgets visitors whose window has expired so memory stays bounded.	10
`clientAddress(req, env = process.env)`	The caller's address. Behind a reverse proxy every connection comes from the proxy, so the proxy's forwarded address is used only when the operator says the proxy is trusted (BOARDROOM_TRUST_PROXY=1).	34

shared/

`shared/board.js`

Shared by browser and server: reads frames, code and input out of Excalidraw board elements.

Function	What it does	Line
`framesIn(elements)`	Lists the frames on the board.	4
`isOverlay(element)`	True for highlight elements the app drew, so they are never treated as writing.	8
`extractProgram(elements, frameId)`	Raw text and blank lines are preserved: compiler line N must remain board line N.	13
`programSignature(program, frameId)`	A fingerprint of the program and frame, used to tell if results are stale.	42
`diagnosticBoxes(diagnostics, lineMap)`	Converts diagnostic line numbers into rectangles on the board.	48

`shared/handwriting.js`

Shared: detects handwriting in a frame, fingerprints ink and builds the OCR review draft.

Function	What it does	Line
`channelElements(elements, frameId, channel)`	Returns the ink elements of one channel (code or input) in a frame.	3
`handwritingChannels(elements, frameId)`	Reports whether the frame has handwritten code and/or input.	9
`inkFingerprint(elements, frameId, channels = ['code', 'stdin'])`	A fingerprint of the ink, so a confirmation is invalidated if strokes change.	15
`mappedLine(box, bounds, elementIds)`	Maps an OCR line box back onto board coordinates.	26
`makeReviewDraft(recognition, captures, typed, fingerprint, frameId)`	Builds the editable review draft from the model's result and the captures.	38
`reviewedProgram(draft)`	Turns a confirmed draft into the program (source and input) that will run.	56

`shared/languages.js`

Shared: the list of supported languages (C++ and Python) and their request fields.

Function	What it does	Line
`validLanguage(value)`	True for a supported language (or none, meaning C++).	6
`languageInfo(value = 'c')`	Label, file name and settings for a language.	7
`languagePayload(value)`	The language field to include in an API request.	8

src/

src/main.jsx

The whole browser application shell: state, the run button, panels, tabs and every user action.

Function	What it does	Line
`Icon({ name, size = 18 })`	Draws one of the app's small SVG icons by name.	36
`MemoryDiagram({ stdout })`	Shows the stack-pointer and heap addresses printed by the memory lesson.	62
`OwnerKeyForm({ onSaved })`	Shown only to the site owner's browser when the API is locked: the key is checked by the server, not here.	76
`App()`	The root component: holds all state and renders the board, toolbar and side panel.	84
`refreshCanvas()` `inside App`	Asks Excalidraw to redraw the board.	168
`async checkHealth()` `inside App`	Asks the API what is available (runner, OCR, history, owner key) and updates the UI.	172
`changed()` `inside App`	Keeps the fullscreen state in sync with the browser.	208
`leave(event)` `inside App`	Escape key: cancels region selection and closes tool menus.	213
`onChange(next, appState)` `inside App`	Called by Excalidraw on every edit; stores the elements and finishes a region selection.	232
`resetWork()` `inside App`	Clears OCR confirmation, results, errors and editor edits.	247
`chooseLesson(id)` `inside App`	Loads a demo lesson onto the board.	254
`changeFrame(id)` `inside App`	Switches the selected program frame and zooms to it.	268
`editCode(value)` `inside App`	Records hand edits in the code editor without changing the board.	275
`async requestJson(path, body, timeoutMs)` `inside App`	POSTs JSON to the API with a timeout and returns the parsed reply.	281
`scanInput()` `inside App`	Opens the handwriting tab and reads only the blue input ink.	298
`async run(confirmedSource)` `inside App`	The run button: makes sure handwriting is confirmed, prepares a snippet if Snippet mode is on, otherwise compiles and runs and saves a snapshot.	303
`boardUnchanged()` `inside run`	Checks the board did not change while a request was in flight.	331
`async compile(source, transformed)` `inside run`	Sends the program to /api/run, shows the result and saves the lesson snapshot.	339
`confirmSnippet()` `inside App`	The "Confirm & run" button: runs the prepared snippet program.	366
`toggleSnippet()` `inside App`	Turns Snippet mode on or off.	372

Function	What it does	Line
`boardScene()` inside App	Builds the saveable board scene with its result and OCR review.	380
`download()` inside App	Downloads the board as an Excalidraw file.	394
`async saveSnapshot()` inside App	The Snapshot button: saves the board to lesson history.	405
`async pushSnapshot(scene, title)` inside App	Sends a scene to /api/snapshots.	424
`async saveCompilationSnapshot(scene, execution)` inside App	Automatically saves the board after each completed run.	432
`restoreBoard(data)` inside App	Loads a saved or imported scene back onto the board.	445
`async importBoard(event)` inside App	Reads an .excalidraw file chosen by the user.	462
`async viewLibrary(file = libraryFile)` inside App	Opens the Libraries tab and loads a header's source.	474
`async toggleFullscreen()` inside App	Enters or leaves browser fullscreen.	496
`focusDiagnostic(diagnostic)` inside App	Scrolls the board to the line an error points at.	503
`newBlankBoard()` inside App	Clears the board for a new lesson.	508
`selectCodeRegion()` inside App	Starts dragging a box around code and input.	520
`cancelRegion()` inside App	Cancels region selection.	532
`transcribeWholePage()` inside App	Puts a frame around all writing on the board in one step.	538
`selectPen(kind, customColor = null)` inside App	Chooses the code (black) or input (blue) pen.	552
`zoomBoard(delta)` inside App	Zooms the board in or out.	563
`resetZoom()` inside App	Fits the board back into view.	570
`onSecondaryChange(next, appState)` inside App	Edit handler for the second board in dual-board mode.	580
`toggleDualMode()` inside App	Shows two boards side by side, or one.	587
`changeDualBoards(firstId, secondId)` inside App	Chooses which tabs appear in dual-board mode.	622
`switchTab(targetId)` inside App	Switches to another board tab, saving the current one.	640
`addNewTab()` inside App	Creates a new board tab.	662
`closeTab(e, closeId)` inside App	Closes a board tab.	683

Function	What it does	Line
`closeTools()` `inside App`	Closes the drawing toolbar.	706
`chooseTool(type)` `inside App`	Selects a drawing tool (pen, eraser, shape and so on).	710
`toggleLock()` `inside App`	Keeps the current tool selected after each use.	718
`chooseWidth(width)` `inside App`	Sets the stroke width.	724
`chooseColor(color)` `inside App`	Sets the ink colour.	729
`toggleTheme()` `inside App`	Switches light and dark mode and remembers it.	733
`changeLanguage(next)` `inside App`	Switches between C++ and Python, clearing anything that no longer matches.	744
`async mount()`	Loads the board font, then starts the React app.	981

`src/components/CodeEditor.jsx`

The editable "code that runs" box with error underlines.

Function	What it does	Line
`CodeEditor(props, ref)`	A textarea over a mirrored layer: the mirror draws wavy underlines under error lines.	6
`syncScroll()` `inside CodeEditor`	Keeps the error underlines aligned while the text scrolls.	32

`src/components/DrawToolbar.jsx`

The drawing toolbar above the board (pen, eraser, shapes, widths, colours).

Function	What it does	Line
`ToolIcon({ name })`	Draws one toolbar icon.	40
`ToolButton({ name, label, active, onClick, ...rest })`	One toolbar button with its icon and label.	44
`DrawToolbar({ activeTool, locked, strokeWidth, color, onTool, onLock, onStrokeWidth, onColor, onClose )`	A docked-at-top drawing bar that drives Excalidraw's own tools; it exists only while drawing tools are open.	51
`close(event)` `inside DrawToolbar`	Closes an open toolbar menu when clicking elsewhere.	56

`src/components/FrameControls.jsx`

Width/height controls for the selected program frame.

Function	What it does	Line
`FrameControls({ frame, disabled, onResize, onFit })`	Inputs to resize the frame and a button to fit it on screen.	3

`src/components/HandwritingPanel.jsx`

The handwriting tab: reads ink with the model and lets the teacher correct and confirm it.

Function	What it does	Line
`HandwritingPanel(props, ref)`	The handwriting review panel component.	8

Function	What it does	Line
`async read(mode = 'all')` inside `HandwritingPanel`	Captures the ink and sends it to /api/ocr.	24
`edit(channel, transform)` inside `HandwritingPanel`	Changes the draft (edit, add or remove a line).	82
`confirm(saveCase = false)` inside `HandwritingPanel`	Confirms the reviewed text so it can run, or saves scanned input as a test case.	88

src/components/ProgramRegions.jsx

Draws labels over the program regions on the board.

Function	What it does	Line
`ProgramRegions({ api, frames, selectedId })`	Selection is a view decoration, so it never becomes ink or part of an exported lesson.	4

src/components/SnapshotHistory.jsx

The History tab: lists saved lessons, opens them and shows sync status.

Function	What it does	Line
`SnapshotHistory({ onOpen })`	The lesson history list component.	4
`async refresh()` inside `SnapshotHistory`	Loads the latest saved lessons.	10
`async refreshSync()` inside `SnapshotHistory`	Loads the archive sync status.	20
`async retrySync()` inside `SnapshotHistory`	Asks the server to push lessons now.	27
`async open(id)` inside `SnapshotHistory`	Opens a saved lesson on the board.	40
`async inspect(snapshot)` inside `SnapshotHistory`	Compares a lesson's saved headers with today's versions.	50

src/components/TestCases.jsx

The Cases tab: up to 20 inputs with expected results.

Function	What it does	Line
`TestCases(props, ref)`	The test-case list component.	7
`async importSuite(event)` inside `TestCases`	Loads test cases from a JSON file.	41
`exportSuite()` inside `TestCases`	Downloads the test cases as JSON.	50
`change(id, field, value)` inside `TestCases`	Edits one field of one case.	56
`async run()` inside `TestCases`	Runs every case in order and compares results.	61

src/components/WorkspaceLayout.jsx

Splits the screen into the board and a resizable side panel.

Function	What it does	Line
`WorkspaceLayout({ children, focused, onResize })`	Lays out the board and the side panel.	3
`resize(next)` inside `WorkspaceLayout`	Changes the side panel width (mouse or keyboard).	21

src/lib/api-response.js

Parses API responses safely and turns hosting errors into readable messages.

Function	What it does	Line
`async readApiJson(response)`	Hosts can return HTML or plain text when a function fails before our handler responds. Never expose that body, or mistake it for the API's JSON contract.	3
`responseError(response)`	Builds a readable error for timeouts and non-JSON hosting pages.	12

src/lib/api.js

The single place the browser calls the API from; attaches the owner key.

Function	What it does	Line
`setOwnerKey(value)`	Saves or clears the owner key in this browser.	10
`ownerKey()`	Reads the saved owner key.	17
`apiFetch(path, options = {})`	Calls the API, adding the owner key and never sending cookies.	21

src/lib/canvas.js

Helpers that change the board: lessons, highlights, frames and regions.

Function	What it does	Line
`lessonScene(lesson)`	Builds the board elements for a demo lesson.	4
`clearOverlays(api)`	Removes error highlights from the board.	23
`showDiagnostics(api, diagnostics, lineMap)`	Draws highlights for errors on their board lines.	27
`focusSourceLine(api, line)`	Scrolls to a source line on the board.	32
`replaceChannel(api, frameId, ids, text, color)`	Replaces the typed code or input text inside a frame.	41
`growFrame(elements, frameId, width, height)`	Calculates a larger frame size that never clips content.	66
`resizeProgramFrame(api, frameId, width, height)`	Resizes the selected frame.	76
`fitProgramFrame(api, frameId)`	Zooms the board to fit the selected frame.	82
`isInk(el)`	True for pen strokes and text that count as writing.	87
`centreIn(el, rect)`	An element belongs to a region when its centre is inside it, so a stray edge stroke is not pulled in.	89
`inkBounds(api, padding = 32)`	The rectangle around all writing, with padding.	95
`createRegionFrame(api, rect)`	A selected region becomes its own frame, so every later step keeps working on "one frame = one program".	103

src/lib/capture.js

Exports handwritten ink in a frame as PNG images for OCR.

Function	What it does	Line
`async captureHandwriting(api, elements, frameId, selectedChannels = ['code', 'stdin'])`	Exports the code ink and input ink of a frame as separate PNGs on a white background.	4

src/lib/linked-list-drawing.js

Turns a drawn integer chain such as 1->3->4 into a real C++ linked-list program.

Function	What it does	Line
`linkedListDrawing(source)`	Interpret only an entire, unbranched integer chain, never C++ expressions/code.	2
`linkedListProgram(drawing)`	Generates the C++ program that builds and prints the drawn list.	10
`composeBoardSource(source, language = 'c')`	A drawn integer chain becomes a real linked list; every other text is returned unchanged.	31

src/lib/test-cases.js

Data rules for test cases: validation, storage key and output comparison.

Function	What it does	Line
`emptyCase(id)`	A new blank test case.	6
`suiteKey(language = 'c')`	Browser storage key for a language's cases.	7
`validateSuite(value, language = 'c')`	Checks an imported test-case file.	8
`normalizeOutput(value, comparison = 'exact')`	Prepares output for comparison (exact, trimmed or whitespace-normalised).	22
`compareOutput(expected, actual, comparison = 'exact')`	Compares expected and actual output and finds the first difference.	27

api/

api/health.js

Vercel function: GET /api/health.

Function	What it does	Line
`async handler(req, res)`	Answers GET /api/health: runner, OCR and archive availability, and whether the caller holds the owner key.	8

api/ocr.js

Vercel function: POST /api/ocr.

Function	What it does	Line
`async handler(req, res)`	Checks access, then runs handwriting recognition.	6

api/run.js

Vercel function: POST /api/run (runs code on Wandbox).

Function	What it does	Line
`async handler(req, res)`	Checks access, then runs the program on Wandbox.	6

api/session.js

Vercel function: /api/session, kept for older clients.

Function	What it does	Line
`handler(req, res)`	Compatibility endpoint for clients upgrading from cookie authentication.	4

api/snapshots.js

Vercel function: lesson history backed by GitHub.

Function	What it does	Line
`createSnapshotHandler(createStore = createGithubSnapshotStore)`	Creates the lesson-history function: list, read, save and compare headers via GitHub.	5

`api/snippet.js`

Vercel function: POST /api/snippet.

Function	What it does	Line
`async handler(req, res)`	Checks access, then prepares a snippet program.	6

runner/

`runner/runner.py`

Runs inside the Docker container: compiles and executes the code and turns the output into JSON.

Function	What it does	Line
`strip_comments(source)`	Removes comments and string contents so scans only see real code.	32
`validate_source(source, language="c")`	Checks size and that every #include is on the allowed list.	39
`runtime_limits()`	Sets CPU, file and process limits for the program just before it starts.	57
`run_phase(command, cwd, stdin="", timeout=5, runtime=False, detect_leaks=True)`	Runs one step (compile or execute) with stdin, a deadline and an output cap.	63
`parse_diagnostics(compiler_output, runtime_output)`	Turns gcc and sanitizer messages into structured errors with line numbers.	101
`teaching_notes(source)`	Adds rule-based notes for calls like malloc, free and scanf.	134
`python_diagnostics(output)`	Turns Python syntax errors and tracebacks into structured errors.	181
`execute_python(source, stdin, result)`	Syntax-checks and runs a Python program.	188
`execute(payload, lib=LIB, *, detect_leaks=True)`	Main entry: validates, compiles with sanitizers, runs and returns one JSON result.	209

ocr-service/

`ocr-service/service.py`

The Python model server: loads Qwen2-VL once and answers /ocr, /wrap and /health.

Function	What it does	Line
`load_model()`	Loads Qwen2-VL and its processor once and keeps them in memory.	48
`split_generated(pieces)`	Group (token_text, probability) pairs into lines of (text, min probability).	60
`clean_lines(lines)`	Drop Markdown fences the model may add and trailing blank lines; keep inner blanks.	76
`transcribe(image, channel)`	Sends one image to the model with the transcription prompt and returns lines with uncertainty flags.	86
`wrap_snippet(source)`	Ask the model to add boilerplate around loose C++ lines. The caller verifies the lines were kept.	126
`decode_png(encoded)`	Decodes base64 and checks it is a PNG within the size limit.	140
`_send(self, status, body)` inside Handler	Writes a JSON response.	155
`do_GET(self)` inside Handler	Answers /health with readiness and model name.	163
`do_POST(self)` inside Handler	Answers /ocr and /wrap, one inference at a time.	168
`log_message(self, fmt, *args)` inside Handler	Silences request logging so no content is printed.	196

scripts/

scripts/dev.mjs

What npm run dev starts: the API and the Vite website together.

Function	What it does	Line
`stop(code = 0)`	Stops both the API and the website processes.	9

scripts/ocr-benchmark-metrics.mjs

Error-rate maths for the optional OCR benchmark.

Function	What it does	Line
`distance(left, right)`	Edit distance between two strings.	3
`scoreTranscript(expected, actual)`	Character, line and symbol error rates for one sample.	25
`normalized(value)` `inside scoreTranscript`	Normalises line endings before scoring.	26
`summarizeScores(scores)`	Averages scores across samples.	46